

Tree-Based Methods

Overview

- **tree-based methods used for regression and classification**
- **stratifying and segmenting the predictor space into simple regions**
- **bagging, random forests, and boosting**

Reading

An Introduction to Statistical Learning: Chapter 8 (textbook)

<https://www.statlearning.com/>

Decision Trees: Statistical Learning (slideshow)

https://www.reisanar.com/slides/islr-ch_8-notes.pdf

Terms

tree-based methods

family of supervised learning methods that make predictions by recursively splitting the predictor space

decision tree

model represented as a tree of sequential splitting rules where each path leads to a decision

regression tree

tree used when the response variable is quantitative
predictions are typically the mean response within a region

classification tree

tree used when the response variable is qualitative or categorical
predictions are the most common class within a region

predictor/feature space

space formed by the input variables
tree methods divide this space into separate regions

region/box

partitions of the predictor space created by the tree's splits
each region gets its own prediction

split

rule that divides the data into two parts based on a predictor and a threshold
ex: $X_1 \leq t$

binary split

creates exactly two child regions from one parent region

recursive binary splitting

top-down, greedy tree-building procedure that repeatedly chooses the split that most improves the fit at the current step

top-down approach

algorithm that begins with the full dataset and repeatedly divides it into smaller subsets

greedy approach

algorithm that chooses the best immediate split at each step rather than searching all possible full-tree structures

terminal node (leaf node)

end node of the tree

once an observation lands there, the model outputs that node's prediction

internal node

non-terminal node where a split is made on one predictor

stopping criterion

rule for deciding when tree growth should stop

ex: when a node has too few observations remaining to split further

prediction surface

function implied by the tree over the feature space

for regression trees it is piecewise constant across regions

Residual Sum of Squares (RSS)

the criterion used to measure fit

splits are chosen to reduce RSS

mean response in a region

the prediction for a region is the average of the training responses inside that region

overfitting

training data is fit too closely and performs poorly on new data

large trees are especially prone to this

pruning

cutting back a large tree to a smaller subtree to improve generalization and interpretability

cost-complexity pruning (weakest-link pruning)

pruning method that balances training fit against tree size using a penalty term

subtree

smaller tree obtained by pruning branches from a larger tree

tuning parameter (α)

non-negative parameter in cost-complexity pruning that controls the trade-off between goodness of fit and tree complexity

tree complexity

generally measured by the number of terminal nodes or splits
more complex trees fit training data better but can overfit

cross-validation

resampling method used to estimate test error and choose the best tuning parameter

ex: α

k-fold cross-validation

cross-validation procedure that splits data into K folds, trains on $K - 1$, tests on the held-out fold, and averages the error

mean squared error (MSE)

common regression error metric used to evaluate predictive performance

qualitative response

categorical target variable used in classification trees

ex: (yes/no)

quantitative response

numeric target variable, such as salary or price, used in regression trees

classification error rate

fraction of observations in a node that do not belong to the majority class

class proportion ($\hat{p}^{mk}$)

estimated proportion of training observations in node m that belong to class k

node purity

measure of how dominated a node is by a single class
purer nodes contain mostly one class

Gini index

classification-tree splitting criterion that measures node impurity
smaller values means more-pure nodes

cross-entropy (deviance)

impurity measure for classification trees
numerically similar to the Gini index in many settings

training data

observations used to build the tree and determine splits

test set performance

how well the tree predicts new unseen data
this is what pruning and cross-validation try to improve

Async	
Bagging	
Bootstrap Aggregating	• another name for ‘bagging’ • general-purpose procedure for reducing the variance of a statistical learning method • bootstrap samples are like independent realizations of the data • amounts to averaging the fits from many independent datasets, which would reduce the variance by a factor of $\frac{1}{B}$ where B is the number of bootstrap samples • when bootstrapping, we replace our dataset by sampling with replacement • when bagging, we average the predictions of a model fit to many bootstrap samples n: total number of instances n': number of instances in each bag ($n' < n$) m: number of bags
When to Use Bagging	• when other methods tend to overfit, bagging reduces the variance of the prediction • the empirical distribution is similar to the true distribution of the samples when n is large
Bootstrapping with Decision Trees	• every time we fit a decision tree to a bootstrap sample, we get a different tree T_b • may lead to loss of interpretability • for each predictor ○ add up the total amount by which the RSS (or Gini index) decreases every time we use the predictor in T_b ○ then, average this total over each bootstrap estimate T₁, ..., T_B
Out-of-Bag Error (OOB)	• every bootstrapped sample has a corresponding hold out or ‘out-of-bag’ sample which is used to test • to determine the test error of a bagging estimate, we could use cross-validation • each time we draw bootstrap sample, we only use 63% of the observations • use the rest of the observations can be used as a test set

Bagging for Regression Trees	• regression trees are grown deep, and are not pruned • each individual tree has high variance, but low bias • averaging these B trees reduces the variance
Bagging for Classification Trees	• for a given test observation, we can record the class predicted by each of the B trees, and take a majority vote • a majority vote is simply the overall prediction is the most commonly occurring class among the B predictions
Variable Importance	• when we bag a large number of trees, it is no longer possible to represent the resulting statistical learning procedure using a single tree • it is no longer clear which variables are most important to the procedure
Complexity of Interpretation	• the collection of bagged trees is much more difficult to interpret than a single tree • an overall summary of the importance of each predictor using ○ RSS (for bagging regression trees) ○ Gini index (for bagging classification trees)
Interpreting Bagging with Regression Trees	• we can record the total amount that the RSS has decreased due to splits over a given predictor averaged over all B trees • a large value indicates an important predictor
Interpreting Bagging with Classification Trees	• add up the total amount that the Gini index is decreased by splits over a given predictor averaged over all B trees

Random Forests	
The Problem with Bagging	- the trees produced by different bootstrap samples can be very similar - random forests provide an improvement over bagged trees by way of a small tweak that de-correlates the trees
The “Small Tweak”	fit a decision tree to different bootstrap samples - when growing the tree, we select a random sample of $m < p$ predictors to consider in each step - this will lead to very different (uncorrelated) trees from each sample - then, average the prediction of each tree
Interpretation	- the optimal m is usually around the square root of p - this can be used as a tuning parameter - the split uses only one of those m predictors - a fresh sample of m predictors is taken at each split
Why Limit Available Parameters?	assume that there is one very strong predictor in the data set along with a number of other moderately strong predictors - in the collection of bagged trees, most or all of the trees will use this strong predictor in the top split all of the bagged trees will look quite similar to each other and the predictions from the bagged trees will be highly correlated averaging many highly correlated quantities does not lead to as large of a reduction in variance as averaging many uncorrelated quantities - this means that bagging will not lead to a substantial reduction in variance over a single tree random forests overcome this problem by forcing each split to consider only a subset of the predictors take note if a random forest is built where $m = p$, then bagging is a special case of random forests

Boosting	
General Approach	• similar to bagging, boosting is a general approach that can be applied to many statistical learning methods for regression or classification trees • booting learns slowly
Process	• use the samples that are easiest to predict • slowly downweigh these cases • then move on to harder samples • fit small (low variance but high bias) trees in each round • boosting learns sequentially and trains on errors • each iteration learns to predict the mistakes of the previous iteration • subsequent samples depend on weights given to records in the previous sample which did not predict correctly (weak learners) • the final prediction is a weighted average
AdaBoost (Adaptive Boosting)	• influential ensemble machine learning method that sequentially combines many simple "weak learners" into a single, high-performing "strong classifier" ○ initialize weights ○ train model in 'usual way' ○ use training data to test model ○ extract error ○ again, randomly select from original data (separate bag) ○ each instance is weighted against error ○ then, build and test model ○ combine run outputs ○ iterate process through m bags

R Programming

- **implementation note: tree vs. rpart in R**
 - both **packages** build **decision** trees.
 - **The main difference is how they handle missing values during splitting and scoring.**
- **tree** package
 - if an observation is **missing** the value for the **primary split variable**
 - it is **not sent** further down the tree
- **rpart** package
 - missing values can be handled in different ways
 - controlled with the **usesurrogate** parameter in **rpart.control**
- **usesurrogate** options
 - **usesurrogate = 0**
 - display **surrogates only**
 - if the **primary split** value is **missing**, the observation is **not split** further
 - this **matches** the behavior of **tree()**
 - **usesurrogate = 1**
 - use **surrogate split rules**, in order
 - if all surrogate **values** are also **missing**, the observation is **not split**
 - **usesurrogate = 2**
 - **use surrogate split rules, in order**
 - if **all** surrogate **values** are **missing**, send the observation in the **majority direction**
 - this is the **default** in **rpart()**

key takeaway

- **usesurrogate = 0** in **rpart** behaves like **tree()**
- **usesurrogate = 2** is the default in **rpart()**
- the default follows the recommendation of Breiman et al. (1984)